

Bachelor's Thesis Proposal

A Web-based Code Editor for Model-driven Privacy

Assessor: Dr. Hoang Nguyen
Co-assessor: To be decided

1 Motivation

Model-driven privacy [2] (MDP) is a software development methodology in which privacy requirements are specified as models, and enforcement mechanisms are generated automatically via model transformations. MDP enables developers to specify three interconnected declarative models: a data model describing the state space of the system, a security model capturing fine-grained access control requirements, and a privacy model expressing data protection requirements. From these models, the methodology automatically generates a web application together with enforcement mechanisms that make it security- and privacy-aware by design.

The authors of the aforementioned paper have built and published the tool, called ν ActionGUI [4], which implements this methodology and generates web applications using the Flask [1] (Python) framework. However, the models are currently written as plain text files, and the tool is run from the command line. Developers, therefore, receive little support while writing the models: there is no syntax highlighting, no code completion, and errors, e.g., a reference to an undefined entity in the security model, are reported only when the generator is run. Moreover, getting started requires a local installation of the tool, which is a barrier for newcomers and for teaching. Modern code editors address these problems with language-aware features, which can be provided in the browser, e.g., with the Monaco editor.

2 Objective

The objective of this thesis is to design and implement a web-based code editor for the model-driven privacy approach. The editor should allow developers to write the data, security, and privacy models directly in the browser, assist them with language-aware features, and generate and download (or run) the resulting web application with ν ActionGUI.

At a minimum, the editor should provide syntax highlighting, validation with error reporting while typing, and code completion for the three models, including references across the models (e.g., from the privacy model to the entities and attributes of the data model).

3 Tasks

1. Study the existing MDP approach and the textual modeling languages of ν ActionGUI.
2. Review technologies for building web-based editors (e.g., the Monaco editor [3]).
3. Design the editor's architecture, including how it may integrate with the existing generator.
4. Implement the editor with syntax highlighting, validation, and code completion for the MDP models, and support for generating the web application.
5. Validate the editor on existing case study models.
6. Document the design, implementation, and evaluation in the final report.

4 Deliverables

The following deliverables are due at the end of the project:

Final report Please have a look at the submission requirements in the following section. The final report should consist of an introduction, a (theoretical) background section, sections describing the work packages and their outcomes, and a conclusion. The report must be written in English.

Source code Complete, documented source code with a reproducible evaluation.

Presentation At the end of the thesis, a 20-minute presentation must be given in a thesis defense seminar. It should give an overview and discuss the most important highlights of the work.

References

- [1] Flask. Flask Web Framework, 2023.
- [2] Srdan Krstic, Hoang Nguyen, and David Basin. Model-driven privacy. *Proc. Priv. Enhancing Technol.*, 2024(1):314–329, 2024.
- [3] Microsoft. Monaco editor, 2024. <https://microsoft.github.io/monaco-editor/>.
- [4] Hoang Nguyen, Srdan Krstic, and David Basin. ν ActionGUI: Model-driven security and privacy, 2024.